MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 3
Operating Systems
Theme: The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
The data set 3
1 MOVING FAST AND FINDING HELP **5**
1.1 Completion and history 5
1.2 Built-in and external commands 5
1.3 Help without a browser 7
1.4 Answers to the questions 8
2 WHAT THE SHELL DOES TO THE COMMAND LINE **9**
2.1 The test files 9
2.2 Predictions and results 9
2.3 Answers to the questions 10
3 REDIRECTION, CHAINING AND JOBS **12**
3.1 Standard output and standard error 12
3.2 The order of redirections 12
3.3 Chaining commands 13
3.4 Background jobs 13
3.5 Appending and reading from a file 13
3.6 Answers to the questions 14
4 ANSWERING QUESTIONS WITH ONE-LINERS **15**
4.1 The tools 15
4.2 The questions 15
4.3 The file oneliners.sh 17
4.4 Bonus: failed requests at a round minute 19
5 THE FIRST SCRIPT **20**
5.1 The script report.sh 20
5.2 Bonus: running the script with cron 21
5.3 Answers to the questions 22
CONCLUSIONS **23**
REFERENCES **24**

INTRODUCTION

Purpose of the work

The purpose of the third laboratory work is to use the command line of Linux quickly and with understanding: to predict what the shell does to a command line before it runs, to redirect the output and the errors of a program where they are needed, and to answer real questions about data with one line of small filters joined by pipes [1]. The shell used in the work is bash, the default shell of Ubuntu [2].

The following tasks were set for the work:

a) to use completion, the command history and the built-in help, and to find out how the shell locates a command;

b) to observe the expansions that the shell performs on a command line: patterns, quotes, braces, arithmetic and command substitution;

c) to redirect the standard output and the standard error, to chain commands and to run a job in the background;

d) to answer eight questions about a web access log with one-line pipelines;

e) to turn the best one-liners into a script with error handling, and to run it periodically with cron.

All commands were run in the terminal of an Ubuntu 26.04 LTS virtual machine in Oracle VirtualBox 7.2, the same machine as in the previous laboratory works. The user is `wancea`.

The data set

Part 4 uses an artificial web access log that is the same for every student, so the answers can be checked. It was generated with the loop from the handout in the folder ~/os-lab3:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -
eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
```

Listing 1 – Generation of access.log

The command `seq 1 500` prints the numbers from 1 to 500, and the loop writes one line for each of them. The operator % gives the remainder of a division, so the minute repeats from 00 to 59, the user from user0 to user6 and the page from /page0 to /page3. Every line whose number divides by 5 gets the result FAIL, all the others get OK. The redirection `> access.log` after `done` sends everything the loop prints into the file. The check `wc -l access.log` printed `500 access.log`, and

the first line of the file is `2026-10-05 10:01 user1 OK /page1`. Each line has five fields separated by spaces: date, time, user, result and page.

1 MOVING FAST AND FINDING HELP

1.1 Completion and history

The key Tab completes the names of commands and files. After `ls /usr/sh` the first Tab completed the line to `ls /usr/share/`, because share is the only entry in /usr that starts with sh. From that point the name was ambiguous, so the next Tab only rang the bell, and one more Tab made bash offer the list of all matches with the question `Display all 252 possibilities? (y or n)`, which was answered with n (Figure 1.1). After `cd /et` a single Tab was enough to complete the name to /etc/, because no other entry of / starts with et. The first attempt at this step failed: the command was sent with Enter instead of Tab, and bash ran `cd /et` literally and answered `No such file or directory`. This shows that the shell never guesses a name; only Tab fills a name in, and only when asked.

```
wancea@OS-Lab1-2:~/os-lab3$ ls /usr/share/
Display all 252 possibilities? (y or n)
wancea@OS-Lab1-2:~/os-lab3$ ls /usr/share/^C
wancea@OS-Lab1-2:~/os-lab3$ cd /etc/
wancea@OS-Lab1-2:/etc$ pwd
/etc
wancea@OS-Lab1-2:/etc$ cd ~/os-lab3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 1.1 – Tab completion

The shortcut Ctrl+R searches the command history backwards. After the commands `date`, `whoami` and `pwd` were run, Ctrl+R followed by the letters da found the earlier `date` command, and Enter ran it again. Bash keeps the history in the file ~/.bash_history.

1.2 Built-in and external commands

When a command is typed, the shell first checks whether it is one of its own built-in commands. If it is not, it searches the folders listed in the variable PATH for a program file with that name. The command `type` shows which case applies (Figure 1.2). The commands cd and echo are shell builtins, while ls is an alias for `ls --color=auto` that runs the program /usr/bin/ls, as `which ls` confirms. The command `type -a echo` shows that echo exists three times: as a builtin and as the programs /usr/bin/echo and /bin/echo, which are the same file, because on Ubuntu /bin is a link to /usr/bin. Bash uses the builtin, because builtins are checked before PATH is searched.

```
wancea@OS-Lab1-2:~/os-lab3$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
wancea@OS-Lab1-2:~/os-lab3$ type -a echo
echo is a shell builtin
echo is /usr/bin/echo
echo is /bin/echo
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 1.2 – Output of type and which

The command cd must be a builtin. Every external program runs as a separate child process of the shell, and every process has its own current directory that only the process itself can change. A cd program would change its own directory and then exit, and the shell would stay where it was. To change the directory of the shell, the code has to run inside the shell.

The variable PATH holds a list of folders separated by colons. The command `echo $PATH | tr ':' '\n'` replaces every colon with a new line, so each folder appears on its own line, and `wc -l` counts them (Figure 1.3). PATH contains 10 directories: /usr/local/sbin, /usr/local/bin, /usr/sbin, /usr/bin, /sbin, /bin, /usr/games, /usr/local/games and /snap/bin, which appears twice because two different startup files add it. The repetition is harmless, since the shell stops at the first folder that contains the command.

```
wancea@OS-Lab1-2:~/os-lab3$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/
games:/snap/bin:/snap/bin
wancea@OS-Lab1-2:~/os-lab3$ echo $PATH | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
wancea@OS-Lab1-2:~/os-lab3$ echo $PATH | tr ':' '\n' | wc -l
10
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 1.3 – The directories in PATH

For a name that exists nowhere, bash searches all of PATH, prints `command not found` and sets the exit code 127 (Figure 1.4). The variable `$?` always holds the exit code of the last command, and the operator ; lets the echo run even though the command before it failed.

```
wancea@OS-Lab1-2:~/os-lab3$ nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 1.4 – Exit code of a missing command

1.3 Help without a browser

The manual page `man ls` describes every option of ls, and a search for /sort inside it leads to the option -t, which sorts the files by modification time, newest first. The same information is printed in short by `ls --help`, whose first lines were kept with `head`; filtering the help text with `grep -i modification` finds the line "Sort by modification time (the 'mtime' in the inode), newest first", which belongs to -t. The command `man -k "list directory"` searches the titles of all manual pages and found ls, dir and vdir, the same programs under the names rust-ls, rust-dir and rust-vdir, and the GNU versions gnuls, gnudir and gnuvdir. This shows a detail of Ubuntu 26.04: its ls is the Rust rewrite of the core utilities, uutils coreutils 0.8.0, as `ls --version` reports, while the GNU versions stay installed under other names. Builtins such as cd have no manual page of their own, because they are part of bash, so bash provides the command `help` for them (Figure 1.5).

```
wancea@OS-Lab1-2:~/os-lab3$ ls --help | head -5
List directory contents.
Ignore files and directories starting with a '.' by default

Usage: ls [OPTION]... [FILE]...

wancea@OS-Lab1-2:~/os-lab3$ ls --help | grep -i modification
          modification time: mtime, modification.
          status change time (the 'ctime' in the inode) instead of the modification
          status access time instead of the modification time. When explicitly
          Sort by modification time (the 'mtime' in the inode), newest first.
wancea@OS-Lab1-2:~/os-lab3$ man -k "list directory"
dir (1)              - List directory contents. Ignore files and directories sta...
gnudir (1)           - list directory contents
gnuls (1)            - list directory contents
gnuvdir (1)          - list directory contents
ls (1)               - List directory contents. Ignore files and directories sta...
ntfsls (8)           - list directory contents on an NTFS filesystem
rust-dir (1)         - List directory contents. Ignore files and directories sta...
rust-ls (1)          - List directory contents. Ignore files and directories sta...
rust-vdir (1)        - List directory contents. Ignore files and directories sta...
vdir (1)             - List directory contents. Ignore files and directories sta...
wancea@OS-Lab1-2:~/os-lab3$ help cd | head -2
cd: cd [-L|[-P [-e]]] [-@] [dir]
    Change the shell working directory.
wancea@OS-Lab1-2:~/os-lab3$ ls --version | head -1
ls (uutils coreutils) 0.8.0
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 1.5 – Help from ls --help, man -k and help cd

1.4 Answers to the questions

1. The commands cd and echo are builtins and ls is an external program (here behind an alias). cd must be a builtin because a child process cannot change the current directory of its parent.

2. PATH contains 10 directories (9 different ones, /snap/bin is listed twice), and ls is in /usr/bin.

3. A command that is not found returns the exit code 127.

4. The option -t of ls sorts by modification time.

2 WHAT THE SHELL DOES TO THE COMMAND LINE

2.1 The test files

A test folder ~/os-lab3/play was created with six empty files (Figure 2.1). The quotes in `touch ... "alfa beta.txt"` make one file whose name contains a space. When ls prints to a terminal, it shows this name in single quotes, as 'alfa beta.txt'. The quotes are not part of the name: ls adds them so that the name can be copied back into a command and still be read as one argument [6].

```
wancea@OS-Lab1-2:~$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
wancea@OS-Lab1-2:~/os-lab3/play$ touch a.c b.c file1.txt file2.txt notes.txt "alfa
beta.txt"
wancea@OS-Lab1-2:~/os-lab3/play$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
wancea@OS-Lab1-2:~/os-lab3/play$
```

Figure 2.1 – Files created in the play directory

2.2 Predictions and results

Before each line was run, a prediction of its output was written down. Table 2.1 compares the predictions with the real outputs (Figures 2.2 to 2.5).

Table 2.1 – Predicted and real outputs of the expansion tests

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	a.c b.c	a.c b.c	As predicted.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	As predicted.
`echo [a-b]*`	a.c b.c	a.c alfa beta.txt b.c	The file alfa beta.txt also starts with a, so the pattern matches it too.
`echo *.xyz`	an empty line	*.xyz	When no file matches, bash leaves the pattern unchanged, and echo prints it as text.
`ls alfa beta.txt`	alfa beta.txt	cannot access 'alfa', cannot access 'beta.txt'	The space splits the name into two arguments, and neither file exists.
`ls "alfa beta.txt"`	alfa beta.txt	'alfa beta.txt'	ls adds quotes around a name with a space when it prints to a terminal.
`echo '$HOME' "$HOME" \$HOME`	$HOME /home/wancea $HOME	$HOME /home/wancea $HOME	As predicted.
`echo {1..5} ; echo $((7 * 6))`	1 2 3 4 5, then 42	1 2 3 4 5, then 42	As predicted.
`echo "today is $(date +%A), I am $(whoami)"`	today is Sunday, I am wancea	today is Sunday, I am wancea	As predicted.

Command	My prediction	Real output	Explanation (if different)
`touch log{1..3}.txt && ls log*`	log1.txt log2.txt log3.txt	log1.txt log2.txt log3.txt	As predicted.

```
wancea@OS-Lab1-2:~/os-lab3/play$ echo *.c
a.c b.c
wancea@OS-Lab1-2:~/os-lab3/play$ echo file?.txt
file1.txt file2.txt
wancea@OS-Lab1-2:~/os-lab3/play$ echo [a-b]*
a.c alfa beta.txt b.c
wancea@OS-Lab1-2:~/os-lab3/play$ echo *.xyz
*.xyz
wancea@OS-Lab1-2:~/os-lab3/play$
```

Figure 2.2 – Globbing results

```
wancea@OS-Lab1-2:~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
wancea@OS-Lab1-2:~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
wancea@OS-Lab1-2:~/os-lab3/play$ ls alfa\ beta.txt
'alfa beta.txt'
wancea@OS-Lab1-2:~/os-lab3/play$
```

Figure 2.3 – The ls error on a name with a space, and the fix

```
wancea@OS-Lab1-2:~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/wancea $HOME
wancea@OS-Lab1-2:~/os-lab3/play$
```

Figure 2.4 – Single quotes, double quotes and backslash

```
wancea@OS-Lab1-2:~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
wancea@OS-Lab1-2:~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"
today is Sunday, I am wancea
wancea@OS-Lab1-2:~/os-lab3/play$ touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
wancea@OS-Lab1-2:~/os-lab3/play$
```

Figure 2.5 – Brace expansion, arithmetic and command substitution

2.3 Answers to the questions

1. The predictions, the real outputs and the explanations are given in Table 2.1.

2. The file names in the output of `echo *.c` were produced by the shell. Before echo starts, bash replaces the pattern *.c with the names of the matching files in the current folder, so echo only receives the finished list a.c and b.c and prints it. echo itself never looks at the files.

3. `ls alfa beta.txt` failed because the shell splits a command line at spaces, so ls received two arguments, alfa and beta.txt, and neither file exists. The first fix is to quote the name, `ls "alfa`

`beta.txt"`. The second fix is to escape the space with a backslash, `ls alfa\ beta.txt`. Both keep the space inside one argument (Figure 2.3).

4. Single quotes keep everything inside literally, so '$HOME' stays $HOME. Double quotes keep the spaces inside one argument but still let the shell expand variables and command substitutions, so "$HOME" becomes /home/wancea. A backslash makes only the next character literal, so \$HOME is printed as $HOME as well.

3 REDIRECTION, CHAINING AND JOBS

3.1 Standard output and standard error

Every program starts with three open channels: 0 is the standard input, 1 the standard output and 2 the standard error. The command `ls /etc /nope` produces both normal output and an error, because /etc exists and /nope does not. With `> out.txt 2> err.txt` the listing went into out.txt and the error message into err.txt, and nothing appeared on the screen (Figure 3.1). The file out.txt has 238 lines: the heading /etc: that ls prints when it receives more than one folder, followed by the entries of /etc. The file err.txt holds the single line `ls: cannot access '/nope': No such file or directory`.

```
wancea@OS-Lab1-2:~/os-lab3/play$ cd ~/os-lab3
wancea@OS-Lab1-2:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
wancea@OS-Lab1-2:~/os-lab3$ wc -l out.txt ; cat err.txt
238 out.txt
ls: cannot access '/nope': No such file or directory
wancea@OS-Lab1-2:~/os-lab3$ head -3 out.txt
/etc:
ModemManager
NetworkManager
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 3.1 – stdout and stderr in separate files

3.2 The order of redirections

The two versions `> all.txt 2>&1` and `2>&1 > all.txt` give different results (Figure 3.2). With the first one nothing appears on the screen and the error is the first line of all.txt. With the second one the error is printed on the screen and all.txt contains only the listing. The reason is that bash applies the redirections from left to right before it starts the program, and `2>&1` copies the place where channel 1 points at that moment; it is a copy, not a lasting link. In the first version channel 1 already points to all.txt when 2>&1 is applied, so both channels end in the file. In the second version channel 1 still points to the screen when 2>&1 is applied, so channel 2 is set to the screen, and only afterwards is channel 1 moved to all.txt.

```
wancea@OS-Lab1-2:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; head -2 all.txt
ls: cannot access '/nope': No such file or directory
/etc:
wancea@OS-Lab1-2:~/os-lab3$ ls /etc /nope 2>&1 > all.txt ; head -2 all.txt
ls: cannot access '/nope': No such file or directory
/etc:
ModemManager
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 3.2 – Comparing the order of redirections

3.3 Chaining commands

In `ls /nope 2> /dev/null || echo "failed with code $?"` the error message went to /dev/null, a special file that discards everything written to it. ls failed with the exit code 2, which ls uses for an argument it cannot access, so the operator || ran the echo. In `mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3` every && continued only because the step before it succeeded, the echo printed `I am in /home/wancea/os-lab3/tmp`, and the last cd ran after ; regardless of the result, so `pwd` afterwards showed /home/wancea/os-lab3 again (Figure 3.3).

```
wancea@OS-Lab1-2:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
wancea@OS-Lab1-2:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/
os-lab3
I am in /home/wancea/os-lab3/tmp
wancea@OS-Lab1-2:~/os-lab3$ pwd
/home/wancea/os-lab3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 3.3 – The ; && || operators

3.4 Background jobs

The command `sleep 100 &` started sleep in the background, and bash printed `[1] 7321`, the job number followed by the process ID, and gave the prompt back at once. The command `jobs` listed the job as Running and `kill %1` sent it the termination signal. The second `jobs` on the same line still showed it as Running, because the process had not ended yet at that instant; right after it bash reported the job as Terminated, and a later `jobs` printed nothing, since the job no longer existed (Figure 3.4).

```
wancea@OS-Lab1-2:~/os-lab3$ sleep 100 &
[1] 7321
wancea@OS-Lab1-2:~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
wancea@OS-Lab1-2:~/os-lab3$ jobs
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 3.4 – Running a background job and stopping it

3.5 Appending and reading from a file

The operator > empties a file before it writes, while >> adds to its end. After `echo first > notes.md ; echo second >> notes.md` the file contains the two lines first and second. The command `wc -l < access.log` printed only the number 500, while `wc -l access.log` printed `500 access.log` (Figure 3.5). When the name is given as an argument, wc opens the file itself, knows its name and prints it. With <, the shell opens the file and connects it to the standard input of wc, so wc receives only the content and never learns the name.

```
wancea@OS-Lab1-2:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat n
otes.md
first
second
wancea@OS-Lab1-2:~/os-lab3$ wc -l < access.log
500
wancea@OS-Lab1-2:~/os-lab3$ wc -l access.log
500 access.log
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 3.5 – Appending to a file and reading a file through the standard input

3.6 Answers to the questions

1. The error message went into err.txt and the listing of /etc went into out.txt, because errors are written to channel 2 and normal output to channel 1.

2. `> all.txt 2>&1` sends both channels into the file, while `2>&1 > all.txt` sends the errors to the screen and only the listing into the file. Redirections are applied from left to right, and 2>&1 copies where channel 1 points at that moment, as explained in section 3.2.

3. With `wc -l access.log` wc opens the file itself and knows its name. With `wc -l < access.log` the shell opens the file and passes only its content through the standard input, so wc has no name to print (section 3.5).

4. The operator ; runs the next command in any case. The operator && runs the next command only if the previous one succeeded, that is, returned the exit code 0. The operator || runs the next command only if the previous one failed, that is, returned a code other than 0.

4 ANSWERING QUESTIONS WITH ONE-LINERS

4.1 The tools

Each question was answered with one command line built from small filters joined by pipes. The pipe | connects the standard output of one program to the standard input of the next. The tools used are grep, which keeps the lines that contain a pattern [3], cut, which keeps selected fields of each line, sort, uniq -c, which merges identical neighbouring lines and counts them, wc -l, head and tail. A frequent pattern is to cut out one field and then run `sort | uniq -c | sort -rn`: sort puts identical values next to each other, uniq -c turns each group into one line with a count, and the last sort orders the groups from the largest count down.

4.2 The questions

Q1. The command `grep -c FAIL access.log` counts the lines that contain FAIL: 100 requests failed (Figure 4.1).

```
wancea@OS-Lab1-2:~/os-lab3$ grep -c FAIL access.log
100
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.1 – Number of failed requests

Q2. The page is the fifth field. `cut -d' ' -f5` keeps it, `sort -u` sorts the values and removes duplicates, and `wc -l` counts the result. Four different pages were requested: /page0, /page1, /page2 and /page3 (Figure 4.2).

```
wancea@OS-Lab1-2:~/os-lab3$ cut -d' ' -f5 access.log | sort -u | wc -l ; cut -d' '
-f5 access.log | sort -u
4
/page0
/page1
/page2
/page3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.2 – Number and list of the different pages

Q3. `cut -d' ' -f5 access.log | sort | uniq -c` counts the requests per page: every page received 125 requests (Figure 4.3). The four counts add up to 500, the number of lines of the log, which confirms that no line was lost.

```
wancea@OS-Lab1-2:~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.3 – Requests per page

Q4. The failed requests are kept with grep, the user field is cut out and counted, and the counts are sorted from the largest. The small program awk then prints only the lines whose count equals the count of the first line, so all users that share the maximum are shown. Two users, user3 and user5, have the most failures, 15 each (Figure 4.4). The failure counts of all seven users (14, 14, 14, 15, 14, 15 and 14) add up to 100, the answer of Q1.

```
wancea@OS-Lab1-2:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c |
 sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
wancea@OS-Lab1-2:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c |
 sort -rn | awk 'NR==1{max=$1} $1==max'
     15 user5
     15 user3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.4 – Users with the most failed requests

Q5. One echo line prints both numbers with two command substitutions: user3 made 72 requests, and 15 of them failed (Figure 4.5).

```
wancea@OS-Lab1-2:~/os-lab3$ echo "user3: $(grep -c user3 access.log) requests, $(gr
ep user3 access.log | grep -c FAIL) failed"
user3: 72 requests, 15 failed
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.5 – Requests and failures of user3

Q6. The order of the filters matters here: first grep keeps the FAIL lines, then `tail -3` keeps the last three of them, and finally `cut -d' ' -f2,3` keeps the time and the user. The last three failed requests are 10:10 user0, 10:15 user5 and 10:20 user3 (Figure 4.6).

```
wancea@OS-Lab1-2:~/os-lab3$ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.6 – Last three failed requests

Q7. The file /etc/passwd holds one account per line, with fields separated by colons; the seventh field is the login shell. `cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn` shows four different shells: 40 accounts use /usr/sbin/nologin, 5 use /bin/false, 2 use /bin/bash and 1 uses /bin/sync (Figure 4.7). Only the 2 accounts with /bin/bash can open an interactive session. The programs nologin and false are given to the system accounts that run services, so that nobody can log in as them, and /bin/sync belongs to the historical account sync, which only writes the disk buffers and exits.

```
wancea@OS-Lab1-2:~/os-lab3$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.7 – Login shells in /etc/passwd

Q8. The prediction was that both commands would count one line per entry of /etc and give the same number. In fact `ls /etc | wc -l` printed 237 and `ls -l /etc | wc -l` printed 238. The difference of exactly one comes from the first line of the long format, `total 1156`, which reports the disk space used by the listed files in blocks of 1 KiB. The short format has no such line, and when its output goes into a pipe, ls prints one name per line (Figure 4.8). The same count of 237 entries explains the 238 lines of out.txt in section 3.1, where the extra line was the heading /etc:.

```
wancea@OS-Lab1-2:~/os-lab3$ ls /etc | wc -l ; ls -l /etc | wc -l
237
238
wancea@OS-Lab1-2:~/os-lab3$ ls -l /etc | head -3
total 1156
drwxr-xr-x  5 root                root                4096 Aug 26 05:30 ModemMan
ager
drwxr-xr-x  8 root                root                4096 Sep 29 12:50 NetworkM
anager
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.8 – Line counts of ls and ls -l

4.3 The file oneliners.sh

All eight answers were saved in the file oneliners.sh, each one under a comment with its question (Listing 2). The file uses the name access.log without a path and contains no numbers computed by hand, so it gives correct answers on a freshly generated log when it is run with `bash oneliners.sh` inside ~/os-lab3 (Figure 4.9). Lines that start with # are comments, which bash skips.

```
# Q1. How many requests failed?
grep -c FAIL access.log

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | wc -l ; cut -d' ' -f5 access.log | sort -u

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c

# Q4. Which user or users have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk
'NR==1{max=$1} $1==max'

# Q5. How many requests did user3 make, and how many of them failed?
echo "user3: $(grep -c user3 access.log) requests, $(grep user3 access.log | grep
-c FAIL) failed"

# Q6. Print the last 3 failed requests, showing only time and user.
grep FAIL access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn

# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
# ls -l prints one extra first line, "total N", the disk blocks used by the
listed files.
ls /etc | wc -l ; ls -l /etc | wc -l ; ls -l /etc | head -1
```

Listing 2 – The file oneliners.sh

```
wancea@OS-Lab1-2:~/os-lab3$ bash oneliners.sh
100
4
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
237
238
total 1156
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.9 – Full run of oneliners.sh

4.4 Bonus: failed requests at a round minute

With extended regular expressions, `grep -E` can describe the minute that ends in 0. The pattern `:[0-9]0 .*FAIL` means a colon, any digit, the digit 0 and a space, followed later on the same line by FAIL. The command printed the matching lines, the first five of which are shown in the figure, and the same pattern with `-c` counted them: 50 requests failed at a minute that ends in 0 (Figure 4.10). This agrees with the way the log was built: a line fails when its number divides by 5, and its minute ends in 0 when the number divides by 10, so exactly half of the 100 failures qualify.

```
wancea@OS-Lab1-2:~/os-lab3$ grep -E ':[0-9]0 .*FAIL' access.log | head -5
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
wancea@OS-Lab1-2:~/os-lab3$ grep -cE ':[0-9]0 .*FAIL' access.log
50
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 4.10 – Failed requests at a minute ending in 0

5 THE FIRST SCRIPT

5.1 The script report.sh

The best one-liners were turned into the script report.sh (Figure 5.1). It takes the name of the log as its first argument $1, checks that the file exists, prints the total number of requests and the number of failures, and then a loop prints the number of failures of every user from user0 to user6. In the loop, `{0..6}` gives the values of i, the double quotes in "user$i" let the shell put the number into the pattern, and `$( ... )` stores the result of the pipeline in the variable n.

```
wancea@OS-Lab1-2:~/os-lab3$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
  echo "no such file: $log" >&2
  exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# for each of user0 ... user6, print "userN: X failures"
for i in {0..6}; do
  n=$(grep "user$i" "$log" | grep -c FAIL)
  echo "user$i: $n failures"
done
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 5.1 – Contents of report.sh

The first run, before the file was made executable, was refused with `Permission denied`. To show this again for the figure, the permission was removed with `chmod -x` and the run was refused in the same way. After `chmod +x report.sh` the listing shows the x bits in `-rwxrwxr-x`, and the script printed 500 requests and 100 failed, followed by the counts per user, which add up to 100 (Figure 5.2).

```
wancea@OS-Lab1-2:~/os-lab3$ chmod -x report.sh ; ./report.sh access.log
bash: ./report.sh: Permission denied
wancea@OS-Lab1-2:~/os-lab3$ chmod +x report.sh ; ls -l report.sh
-rwxrwxr-x 1 wancea wancea 365 Oct 11 16:56 report.sh
wancea@OS-Lab1-2:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 5.2 – Running the script on access.log

For a file that does not exist the script printed `no such file: nope.log` and ended with the exit code 1 (Figure 5.3). During the work there was also an instructive mistake: a first version of the file contained only the loop, without the line `log=$1` and without the check. That version printed `grep: : No such file or directory` for every user and ended with the code 0. The variable log had been set in the terminal, but the script runs in a new bash process, and ordinary shell variables are not passed to child processes, so log was empty inside the script.

```
wancea@OS-Lab1-2:~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 5.3 – Error handling for a missing file

5.2 Bonus: running the script with cron

The service cron runs commands on a schedule. The line below was installed as the crontab of the user (Listing 3). `crontab -e` opens this table in an editor; the same result was reached with `echo '<line>' | crontab -`, which reads the new table from the standard input. The five stars stand for the minute, the hour, the day of the month, the month and the day of the week, and a star means every value, so the job runs every minute. Full paths are needed because cron starts jobs with a minimal PATH and in the home folder, and `>> ... 2>&1` appends both the output and the errors to the log [4].

```
* * * * * /home/wancea/os-lab3/report.sh /home/wancea/os-lab3/access.log >>
/home/wancea/os-lab3/report.log 2>&1
```

Listing 3 – The crontab line

The crontab was installed at 17:25. Two minutes later `crontab -l` still showed the line, and the file report.log contained the output of two runs, made at 17:26 and 17:27, each with the total, the number of failures and the seven lines per user (Figure 5.4). The line was then removed with `crontab -r`, and `crontab -l` confirmed that no crontab was left.

```
wancea@OS-Lab1-2:~/os-lab3$ crontab -l
* * * * * /home/wancea/os-lab3/report.sh /home/wancea/os-lab3/access.log >> /home/w
ancea/os-lab3/report.log 2>&1
wancea@OS-Lab1-2:~/os-lab3$ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
wancea@OS-Lab1-2:~/os-lab3$
```

Figure 5.4 – The crontab line and the log written by cron

5.3 Answers to the questions

1. The first line #!/bin/bash, called the shebang, tells the kernel which program must run the file: when ./report.sh is started, the kernel sees the characters #! and actually runs /bin/bash with the script as its argument. Before chmod +x the file does not have the execute permission, so the kernel refuses to run it and bash reports Permission denied.

2. For a name without a slash, bash only searches the folders in PATH, and the current folder is not one of them. This is deliberate: otherwise a program with a common name, such as a fake ls placed in some folder, could be run by accident. The prefix ./ gives the exact path of the file in the current folder.

3. The error is written with >&2 to the standard error, so it stays separate from the real results: if the output of the script is redirected to a file, the message still appears on the screen instead of being hidden in the file. The command exit 1 ends the script with a non-zero code, so $?, the operators && and ||, other scripts and cron can see that the script failed. Without it the script would end with the code of its last command and report success.

CONCLUSIONS

The work showed that the shell is an ordinary program that rewrites every command line before anything runs. Patterns such as *.c, variables such as $HOME, braces, arithmetic and command substitutions are all replaced by bash, quotes are removed, and the line is split into words at the spaces. The program receives only the finished list of arguments. This explains why echo can print file names without knowing anything about files, why a name with a space must be quoted, and why `echo *.xyz` prints the pattern itself when no file matches it.

The second idea of the work is that every program has separate channels for input, output and errors, and that the shell can connect each of them to a file or to another program. This makes it possible to keep the errors apart from the results, to combine them when needed, and to build pipelines in which every tool does one small job. Eight questions about the access log were answered this way with grep, cut, sort, uniq, wc, head and tail, and the counts were checked against each other: the requests per page add up to the 500 lines of the log, and the failures per user add up to the 100 failed requests.

The third idea is the exit code, the number with which every program ends. The operators && and || decide on it, and the script report.sh uses exit 1 to signal a missing file. Two mistakes during the work were useful: a command sent with Enter instead of Tab showed that bash runs exactly what is typed, and a script without the line log=$1 showed that a variable of the terminal does not reach a child process. The same rule about child processes is the reason why cd has to be a builtin of the shell.

REFERENCES

1. The Command Line: Shell, Pipes and Text Filters. Laboratory Work 3, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.

2. GNU Bash Reference Manual. Free Software Foundation [online]. Available: https://www.gnu.org/software/bash/manual/bash.html

3. grep(1). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man1/grep.1.html

4. crontab(5). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man5/crontab.5.html

5. SHOTTS, W. The Linux Command Line. 2nd edition. San Francisco: No Starch Press, 2019.

6. GNU Coreutils manual: ls invocation, formatting the file names [online]. Available: https://www.gnu.org/software/coreutils/manual/html_node/Formatting-the-file-names.html